



전자전기컴퓨터설계실험II

실험 후 보고서

FPGA 응용회로 실습

실험 19-25

학과	전자전기컴퓨터공학부
이름 / 학번	조성현 / 2023440129
작성일	2026년 10월 09일
대상	PWM LED 밝기 · RGB LED PWM · 피에조 단일 음 · 스텝모터 위상 제어 · MM:SS 시계 · 문자 LCD 제어 · PC-FPGA UART 에코
검증환경	VS Code 1.137.0 / Icarus Verilog 13.0 / VaporView / Vivado 2026.1

소스 식별

GitHub 계정: a40093632-ai

GitHub 저장소: <https://github.com/a40093632-ai/ece2-lab3>

제출 태그: [lab3-post-v1](#)

커밋 해시: [0585a5534c32f9107049f9fd63eb75f29e2f8429](#)

1. 실험 목적

본 실험에서는 50MHz 시스템 클록을 사용하는 FPGA 응용회로 7종을 Verilog HDL로 설계하고, RTL 시뮬레이션, Vivado 구현 및 실제 FPGA 보드 시연까지 수행하였다. 각 회로는 VS Code와 Icarus Verilog 환경에서 테스트 벤치로 먼저 검증하고, Vivado XSim에서 동일한 자극과 파형을 확인하였다. 이후 Synthesis, Implementation, Timing Summary 및 DRC를 확인한 뒤 bitstream을 생성하였다. 마지막으로 FPGA 보드에서 버튼, LED, 7-Segment, 피에조, 스텝모터, LCD 및 UART 통신의 실제 동작을 확인하였다. 이를 통해 단일 시스템 클록 기반의 카운터 설계, PWM 듀티비 제어, 상태 전이, 동적 표시, 병렬 LCD 제어 및 UART 직렬 통신의 동작 원리를 이해하고, RTL 설계부터 실제 보드 검증까지의 FPGA 설계 과정을 익히는 것을 목적으로 한다.

2. 관련 이론

순차 제어와 상태 전이

스텝모터, LCD 및 UART 회로는 단순한 조합논리 출력이 아니라 정해진 순서에 따라 상태가 변하는 순차 제어 회로이다. 스텝모터는 여자 패턴을 순서대로 출력해야 회전하고, LCD는 초기화 명령과 문자 데이터를 정해진 순서로 전송해야 하며, UART는 start bit, data bits, stop bit 순서로 프레임 처리해야 한다.

이러한 회로에서는 현재 상태와 카운터 값이 다음 상태 및 출력을 결정한다. 따라서 회로의 정상 동작은 개별 출력값 하나만 확인하는 것이 아니라, 상태 전이 순서와 각 상태의 유지 시간이 올바른지 함께 확인해야 한다. 본 실험에서는 파형에서 상태 변화, 제어신호 및 데이터 전송 순서를 확인하여 순차 제어가 기대한 순서대로 진행되는지 검증하였다.^[1]

외부 입력의 동기화와 신뢰성

버튼 입력과 UART RX 신호는 FPGA의 시스템 클록과 독립적으로 변화하는 외부 입력이다. 외부 입력이 클록 에지 부근에서 변하면 플립플롭이 불안정한 상태가 되는 metastability가 발생할 수 있다. 이를 줄이기 위해 입력을 2단 플립플롭으로 동기화한 뒤 내부 회로에서 사용한다.

버튼은 기계적인 접점의 흔들림으로 짧은 시간 동안 여러 번 변화할 수 있으므로 디바운싱과 one-pulse 처리가 필요하다. UART 수신에서는 start bit를 검출한 뒤 bit 중앙에서 다시 확인하고, 이후 일정한 bit 간격마다 데이터를 LSB-first로 수신한다. 이러한 동기화와 샘플링 과정은 외부 입력을 안정적으로 처리하고, 실제 보드에서 시뮬레이션과 같은 결과를 얻기 위해 필요하다.^[1]

3. 실험 환경 및 공통 검증 방법

실험 환경

구분	내용
운영체제	Windows 11 64-bit
FPGA 설계 도구	AMD Vivado 2026.1
설계언어	Verilog HDL / SystemVerilog
코드 및 사전 검증	VS Code 1.137.0 / Icarus Verilog 13.0 / VaporView
FPGA 소자	xc7s75fgga484-1
회로 종류	FPGA 응용회로
보드	Combo II-DLD S75
시스템 클록	50MHz, 주기 20ns
결과 관리	Git 및 GitHub

공통 실행 순서

1. 각 실험의 RTL 파일은 Vivado Design Source에, 테스트 벤치는 Simulation Sources에 등록.
2. Icarus Verilog로 테스트 벤치를 실행하여 기대값과 PASS 결과를 확인하고, VaporView에서 VCD 파형을 확인.
3. Vivado XSim에서 동일한 테스트 벤치를 실행하여 PASS 결과와 파형이 VS Code 결과와 일치하는지 확인.
4. Synthesis와 Implementation을 수행하고 Design Runs에서 완료 상태를 확인.
5. Utilization, DRC 및 Timing Summary를 확인한 뒤 비트스트림 생성.
6. Hardware Manager에서 FPGA를 프로그램하고 보드의 실제 입력과 출력 동작을 사진 및 영상으로 기록.
7. 소스코드, Vivado 증빙, Bitstream, SHA-256 해시 및 보드 시연 자료를 GitHub에 저장.

실험별 설계파일

실험	Design Top	Simulation Top
19	Lab3_led_pwm	Tb_led_pwm
20	Lab3_rgb_pwm	Tb_rgb_pwm
21	Lab3_piezo	Tb_piezo
22	Lab3_stepper	Tb_stepper
23	Lab3_mmss_clock	Tb_mmss_counter
24	Lab3_character_lcd	Tb_character_lcd
25	Lab3_uart_echo	Tb_uart_echo
실험	XDC	
19	Lab3_led_pwm.xdc	
20	Lab3_rgb_pwm.xdc	
21	Lab3_piezo.xdc	

22	Lab3_stepper.xdc
23	Lab3_mmss_clock.xdc
24	Lab3_character_lcd.xdc
25	Lab3_uart_echo.xdc

장치 프로그램 확인

각 실험에서 Implementation과 bitstream 생성을 완료한 뒤 Hardware Manager를 이용하여 Combo II-DLD S75 FPGA 보드에 프로그램 하였다. Device 상태가 Programmed로 표시되는 것을 확인한 뒤, 실험별 입력장치와 출력장치를 사용하여 실제 동작을 검증하였다.

4. 실험19 결과 · PWM LED 밝기

목적 및 설계

50MHz 클록을 이용하여 1kHz PWM 신호를 생성하고, 버튼을 누를 때마다 LED의 듀티비가 10%씩 증가하도록 설계하였다. 버튼 입력은 2단 동기화, 디바운싱 및 one-pulse 처리를 거쳐 한 번의 버튼 입력당 level이 한 번만 증가한다. 생성된 PWM 신호는 8개의 일반 LED에 동일하게 연결하였다.

Vivado 실험사진



그림 1: 실험 19 Vivado 시뮬레이션 PASS

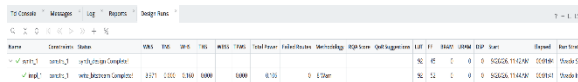


그림 2: 실험 19 합성 및 구현 완료 결과

Summary

Resource	Utilization	Available	Utilization %
LUT	92	48000	0.19
FF	52	96000	0.05
IO	11	338	3.25

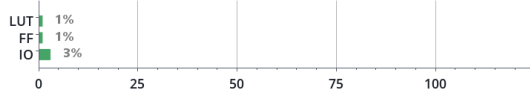


그림 3: 실험 19 FPGA 자원 사용량

Name	Details
Designs (1)	
Designs (1)	

Whether the CS2B5 or CONFG_VOLTAGE property is set in the current design. Configuration bank voltage values (CS2B5 or CONFG_VOLTAGE) must be set to the correct configuration voltage in order to determine the (V) voltage support for the pins in bank 0. It is suggested to specify these either using the Edit Device Properties function in the GUI or directly in the XDC file using the following syntax:

그림 4: 실험 19 DRC 검사 결과

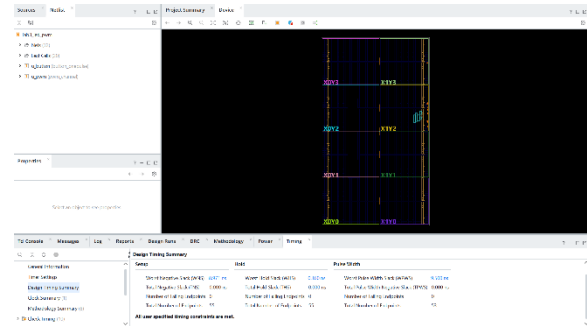


그림 5: 실험 19 타이밍 분석 결과

Algorithm	Hash	Path
SHA256	219E58AF45FFB5C48ED4472F138684A03C5C52EBFF95A8ACCA87B7853C93	D:\Engineering\Lab3\Lab3_pwm_led_pmm\Get-Childitem "D:\Engineering\Lab3\Lab3_pwm_led_pmm\Get-Childitem "Get-FileHash -Algorithm SHA256

그림 6: 실험 19 Bit Hash 확인

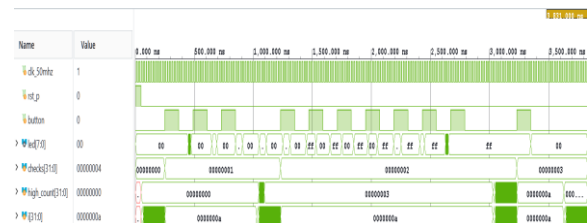


그림 7: 실험 19 입력 조합에 따른 출력 파형

Vivado 시뮬레이션 및 파형 비교

가속 TB에서는 한 PWM 주기를 10클록으로 설정하였다. 따라서 level이 0일 때 HIGH 횟수는 0회, 버튼을 세 번 입력한 level 3에서는 3회, level 10에서는 10회로 나타났다. 최대 밝기 상태에서 버튼을 한 번 더 입력하면 level이 0으로 복귀하고 LED 출력도 다시 꺼졌다. Vivado 파형의 high_count가 0, 3, 10, 0으로 변하여 VS Code에서 확인한 기대 결과와 일치하였다.

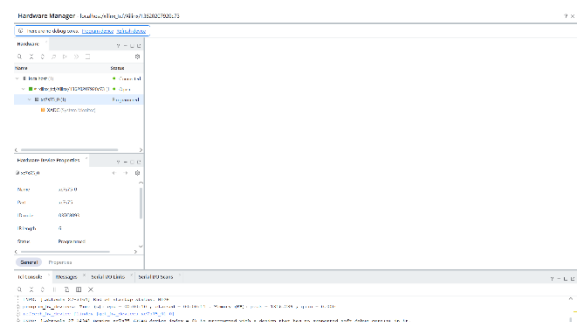


그림 8: 실험 19 Program Device 확인

FPGA 보드 시연

Hardware Manager에서 Device 상태가 Programmed로 표시되는 것을 확인하였다. 버튼을 누를 때마다 8개의 LED 밝기가 단계적으로 증가하였으며, 최대 밝기 다음에는 모든 LED가 꺼지는 초기 상태로 복귀하였다. 사진에서는 8

개의 LED가 동시에 켜진 상태를 확인할 수 있고, 밝기의 단계적 변화는 시연 영상으로 확인하였다.

5. 실험20 결과 · RGB LED PWM

목적 및 설계

R, G, B 세 개의 PWM 채널을 독립적으로 제어하여 RGB LED의 혼합 색상과 밝기를 변경하였다. 각 채널은 별도의 버튼과 level 레지스터를 사용한다.

Vivado 실험사진



그림 9: 실험 20 Vivado 시뮬레이션 PASS

[illegible]

그림 10: 실험 20 합성 및 구현 완료 결과

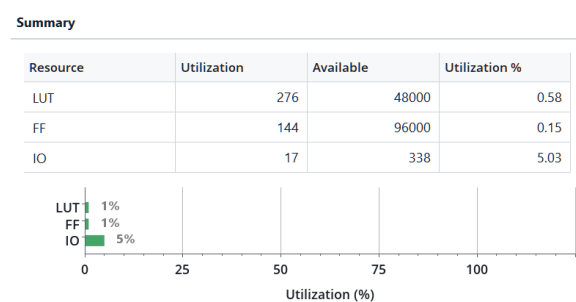


그림 11: 실험 20 FPGA 자원 사용량



그림 12 : 실험 20 DRC 검사 결과

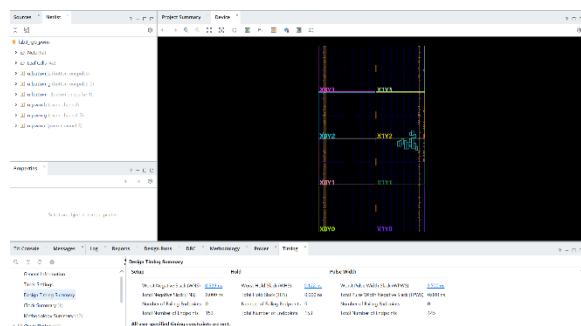


그림 13 : 실험 20 타이밍 분석 결과

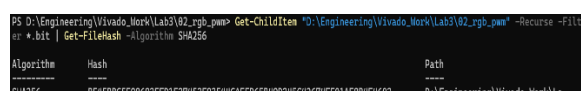


그림 14: 실험 20 Bit Hash 확인

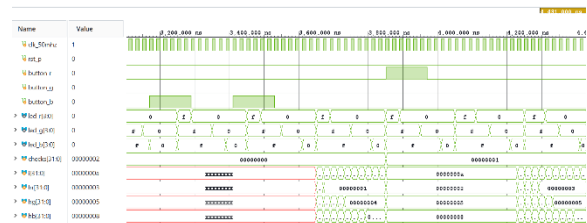


그림 15: 실험 20 입력 조합에 따른 출력 파형

Vivado 시뮬레이션 및 파형 비교

첫 번째 검사에서 R, G, B의 level은 각각 2, 5, 8로 설정되었다. 한 PWM 주기가 10클록이므로 HIGH 횟수는 각각 2회, 5회, 8회로 나타났으며 이는 20%, 50%, 80%의 듀티비에 해당한다. 이후 R 버튼만 한 번 더 입력하였을 때 R의 HIGH 횟수만 3회로 증가하고 G와 B는 각각 5회와 8회를 유지하였다. 따라서 Vivado와 VS Code에서 각 색상 채널의 독립적인 밝기 제어 결과가 동일하게 나타났다.

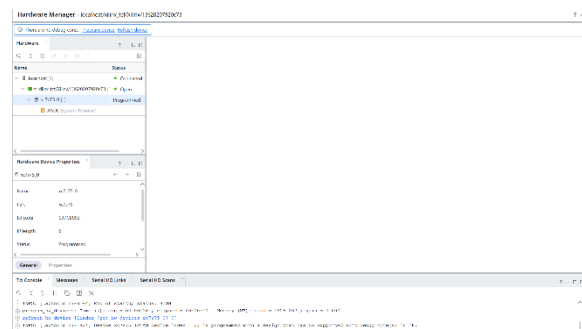


그림 16: 실험 20 Program Device 확인

FPGA 보드 시연

R, G, B 버튼을 각각 누르면 대응하는 색상 채널의 밝기가 독립적으로 증가하였다. 서로 다른 듀티비를 조합하면 Full Color LED에 혼합 색상이 나타났다. 시연 사진에서는 RGB 성분이 혼합된 청록 계열의 출력과 버튼 조작 상태를 확인할 수 있다.

6. 실험21 결과 · 피에조 단일 음

목적 및 설계

50MHz 클럭을 분주하여 294Hz의 사각파를 생성하고 이를 피에조 출력에 연결하였다. 출력은 반주기마다 반전되므로 반주기 카운트는 다음과 같다.

$$\text{HALF PERIOD} = 50,000,000 / (2 \times 294) = 85,034 \text{클럭}$$

한 클럭이 20ns이므로 반주기 시간은 다음과 같다.

$$85,034 \times 20 \text{ ns} = 1.70068 \text{ ms}$$

Vivado 실험사진

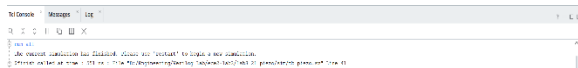


그림 17: 실험 21 Vivado 시뮬레이션 PASS



그림 18: 실험 21 합성 및 구현 완료 결과

Summary

Resource	Utilization	Available	Utilization %
LUT	21	48000	0.04
FF	18	96000	0.02
IO	3	338	0.89

Utilization (%)

그림 19: 실험 21 FPGA 자원 사용량

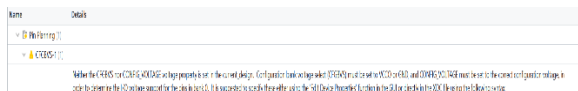


그림 20 : 실험 21 DRC 검사 결과

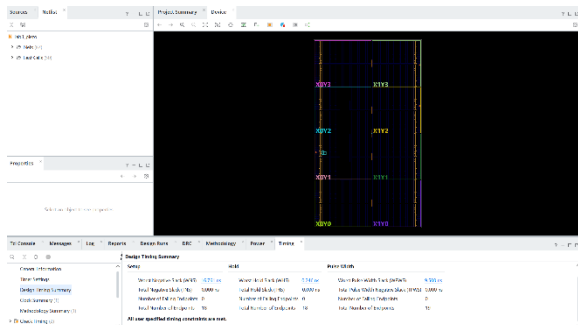


그림 21 : 실험 21 타이밍 분석 결과

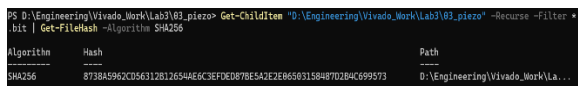


그림 22: 실험 21 Bit Hash 확인

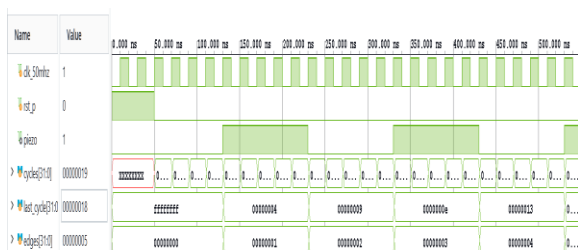


그림 23: 실험 21 입력 조합에 따른 출력 파형

Vivado 시뮬레이션 및 파형 비교

TB는 연속한 출력 에지 사이의 클록 수를 검사하였으며 총 5개의 에지를 정상적으로 확인하였다. 시뮬레이션 결과는 LAB3_PIEZO_PASS edges=5이고 시뮬레이션은 551 ns에 종료되었다. 이는 VS Code에서 확인한 결과와 동일하다. 실제 보드에서는 기본 파라미터가 적용되므로 반주기는 85,034클록이고 약 294 Hz의 단일 음이 출력된다.

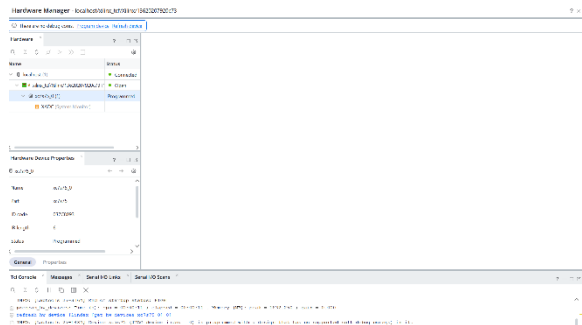


그림 24: 실험 21 Program Device 확인

FPGA 보드 시연

Bitstream을 FPGA에 프로그램하고 보드의 PIEZO ENABLE을 활성화하였다. 피에조에서 일정한 단일 음이 연속적으로 출력되었으며, 음이 끊기거나 불규칙하게 변하지 않았다. 정지 사진만으로는 소리를 확인할 수 없기 때문에 실제 294 Hz 단일 음 출력은 시연 영상을 증빙으로 사용하였다.

7. 실험22 결과 · 스텝모터 위상 제어

목적 및 설계

4상 스텝모터에 순차적인 여자 패턴을 출력하여 회전시키고, enable과 direction 입력으로 정지 및 회전 방향을 제어하였다. 정방향 여자 순서는 0011-0110-1100-1001 순으로 반복된다. 역방향에서는 위 순서를 반대로 진행한다. Enable=0이 되면 위상 상태와 출력을 유지하여 불필요한 상태 전이를 방지한다.

Vivado 실험사진

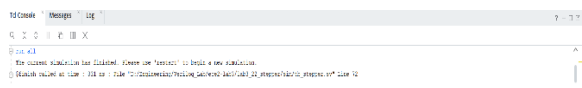


그림 25: 실험 22 Vivado 시뮬레이션 PASS

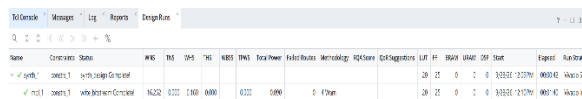


그림 26: 실험 22 합성 및 구현 완료 결과

Summary

Resource	Utilization	Available	Utilization %
LUT	20	48000	0.04
FF	25	96000	0.03
IO	8	338	2.37

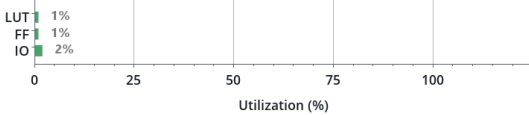


그림 27: 실험 22 FPGA 자원 사용량

[illegible]

그 림 28 : 실험 22 DRC 검사 결과

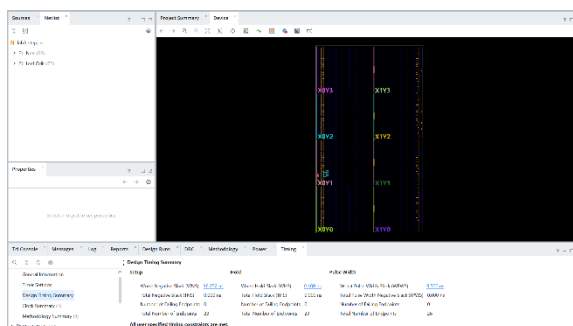


그림 29 : 실험 22 타이밍 분석 결과

```
PS D:\Engineering\ Vivado_Work\Lab3\04_stepper> Get-ChildItem "D:\Engineering\ Vivado_Work\Lab3\04_stepper" -Recurse -File
er *.bit | Get-FileHash -Algorithm SHA256
```

Algorithm	Hash	Path
SHA256	44502656544003241335151008761E1D1C0614F2B333505023334E4E7F694E4E	D:\Engineering\ Vivado_Work\Lab3\04_stepper\04_stepper.bit

그림 30: 실험 22 Bit Hash 확인

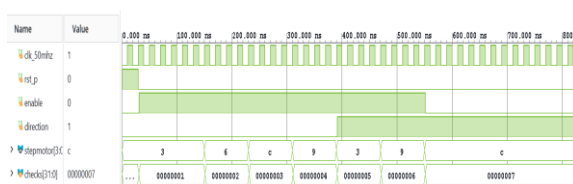


그림 31: 실험 22 입력 조합에 따른 출력 파형

Vivado 시뮬레이션 및 파형 비교

Enable=1, direction=0에서 스텝모터 출력은 0011-0110-1100-1001 순서로 변화하였다. direction을 변경한 뒤에는 여자 패턴이 반대 순서로 진행하였고, enable=0에서는 마지막 출력 상태가 유지되었다. 총 8개의 검사를 통과하여 LAB3_STEPPER_PASS checks=8이 출력되었으며 시뮬레이션은 831 ns에 종료되었다. 상태 전이 순서와 enable 동작이 VS Code 결과와 일치하였다.

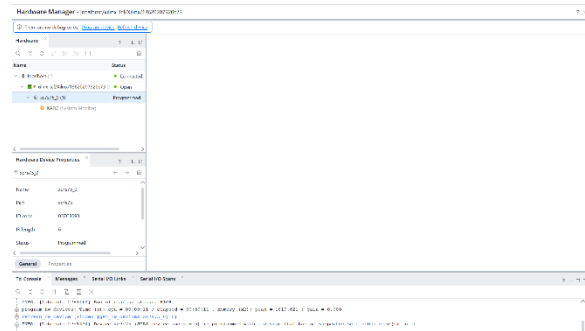


그림 32: 실험 22 Program Device 확인

FPGA 보드 시연

보드에서 enable 버튼을 활성화하자 스텝모터가 일정한 속도로 회전하였다. direction 버튼을 변경하면 모터의 회전 방향이 반대로 바뀌었으며, enable를 비활성화하면 회전이 정지하였다. 보드의 4상 표시 LED에서도 여자 패턴이 순차적으로 바뀌는 것을 확인하였다.

8. 실험23 결과 · MM: SS 시계

목적 및 설계

50MHz 시스템 클럭으로 1초를 생성하고, 분과 초를 각각 BCD 숫자로 계수하여 MM:SS 형식으로 표시하였다. 초 일의 자리는 0~9, 초 십의 자리는 0~5, 분 일의 자리는 0~9, 분 십의 자리는 0~5 범위로 동작한다. 따라서 59:59 다음에는 00:00으로 복귀한다.

Vivado 실험사진

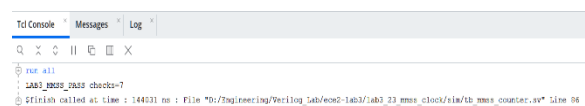


그림 33: 실험 23 Vivado 시뮬레이션 PASS

Name	Comments	Status	MSG	TMS	TMS	TMS	TMS	TMS	TotalPower	Folder/Router	Methodology	PGA score	QoS/Generators	LLT	FF	BSM	LRM	SR	Start	Elapsed
✓ synfs	synfs.1	synfs design Complete																	2020/05/12 14:56	00:01:43
✓ synfs.1	synfs.1	synfs Messages Complete																	2020/05/12 14:56	00:01:36

그림 34: 실험 23 합성 및 구현 완료 결과

Summary

Resource	Utilization	Available	Utilization %
LUT	83	48000	0.17
FF	58	96000	0.06
IO	18	338	5.33

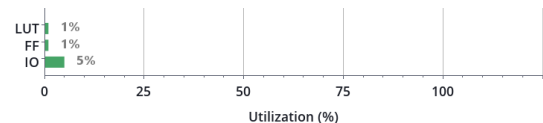


그림 35: 실험 23 FPGA 자원 사용량



그림 36 : 실험 23 DRC 검사 결과

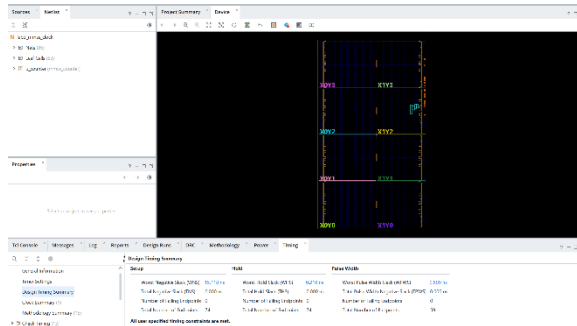


그림 37 : 실험 23 타이밍 분석 결과

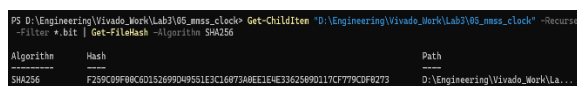


그림 38 : 실험 23 Bit Hash 확인

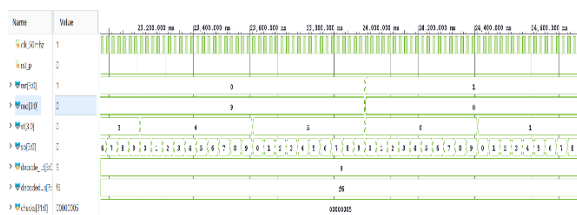


그림 39: 실험 23 입력 조합에 따른 출력 파형

Vivado 시뮬레이션 및 파형 비교

파형에서 초 일의 자리 so가 0부터 9까지 증가한 뒤 0으로 복귀하고, 초 십의 자리 st가 증가하는 것을 확인하였다. 특히 00:59 다음에 01:00으로 정상적인 자리올림이 발생하였다. Vivado에서 LAB3_MMSS_PASS checks=7이 출력되었으며 144031 ns에 종료되어 VS Code 결과와 일치하였다.

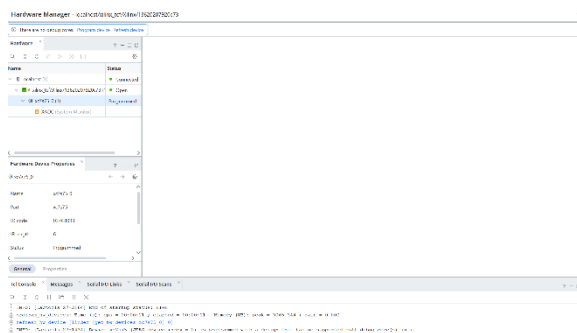


그림 40: 실험 23 Program Device 확인

FPGA 보드 시연

보드의 오른쪽 4개 7-Segment에 MM: SS 값이 표시되었으며, 초가 1초마다 증가하였다. 09초 다음에는 10초로, 59초 다음에는 분이 증가하면서 초가 00으로 복귀하였다. 59:59는 너무 긴 시간이므로 따로 측정하지 않았다.

9. 실험24 결과 · 문자 LCD 제어

목적 및 설계

16x2 문자 LCD에 초기화 명령과 문자 데이터를 순차적으로 전송하여 첫 번째 줄에 FPGA LAB3, 두 번째 줄에 LCD CONTROLLER를 출력하였다. lcd_rw는 항상 0으로 고정하여 쓰기 모드만 사용하였다. lcd_rs=0이면 명령어, lcd_rs=1이면 문자 데이터이다. lcd_e의 enable pulse가 발생할 때 LCD가 lcd_data를 받아들이도록 하였다.

Vivado 실험사진



그림 41: 실험 24 Vivado 시뮬레이션 PASS

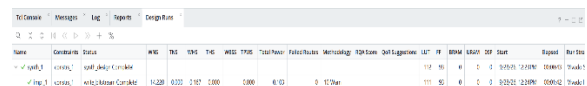


그림 42: 실험 24 합성 및 구현 완료 결과

Summary

Resource	Utilization	Available	Utilization %
LUT	111	48000	0.23
FF	93	96000	0.10
IO	13	338	3.85

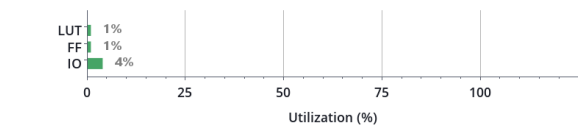


그림 43: 실험 24 FPGA 자원 사용량

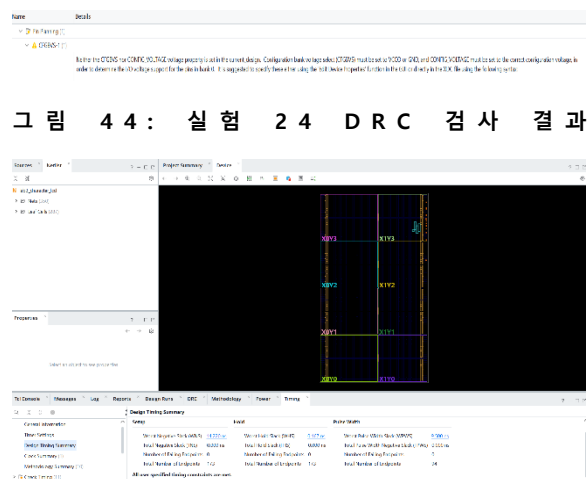


그림 45 : 실험 24 타이밍 분석 결과

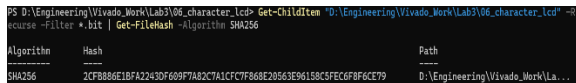


그림 46: 실험 24 Bit Hash 확인

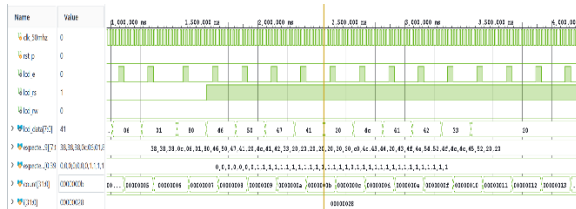


그림 47: 실험 24 입력 조합에 따른 출력 파형

Vivado 시뮬레이션 및 파형 비교

초기화 구간에서는 lcd_rs=0으로 나타나 명령어가 전송되었다. 이때 38, 38, 38, 0C, 06, 01, 80의 초기화 및 첫 번째 줄 주소 명령이 순서대로 출력되었다. 이후 lcd_rs=1로 전환되어 문자 데이터가 전송되었으며, 46, 50, 47, 41, 20, 4C, 41, 42, 33은 ASCII 기준으로 FPGA LAB3에 해당한다. 이어 C0 명령으로 두 번째 줄 주소를 지정한 뒤 LCD CONTROLLER 문자열이 전송되었다.

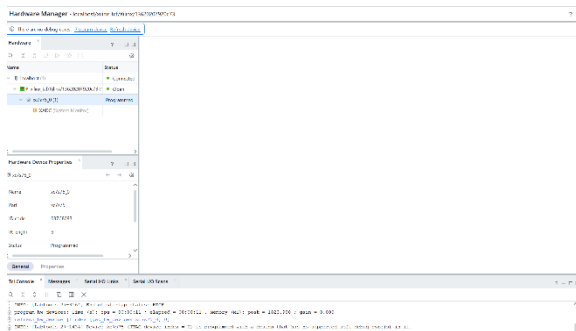


그림 48: 실험 24 Program Device 확인

FPGA 보드 시연

Bitstream을 프로그램한 뒤 LCD 초기화가 완료되자 다음 문자열이 표시되었다. 두 문자열이 올바른 줄과 순서로 출력된 것을 확인하였으며, 문자 누락이나 잘못된 문자는 관찰되지 않았다.

첫 번째 줄: FPGA LAB3,

두 번째 줄: LCD CONTROLLER

10. 실험25 결과 · PC-FPGA UART 에코

목적 및 설계

PC에서 UART로 전송한 8 bit 데이터를 FPGA가 수신하고, 동일한 데이터를 다시 PC로 송신하는 ECHO 회로로 구현

하였다. 마지막으로 수신한 데이터는 8개의 LED에도 출력하였다.

Vivado 실험사진



그림 49: 실험 25 Vivado 시뮬레이션 PASS

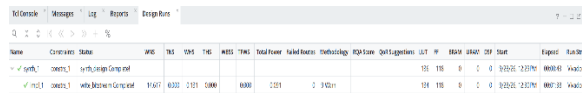


그림 50: 실험 25 합성 및 구현 완료 결과

Summary

Resource	Utilization	Available	Utilization %
LUT	134	48000	0.28
FF	118	96000	0.12
IO	12	338	3.55

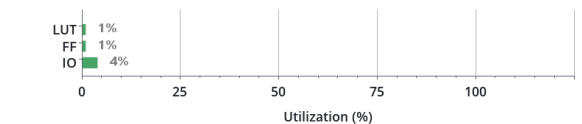


그림 51: 실험 25 FPGA 자원 사용량

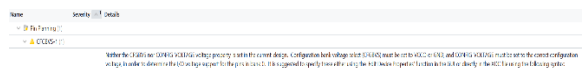


그림 52 : 실험 25 DRC 검사 결과

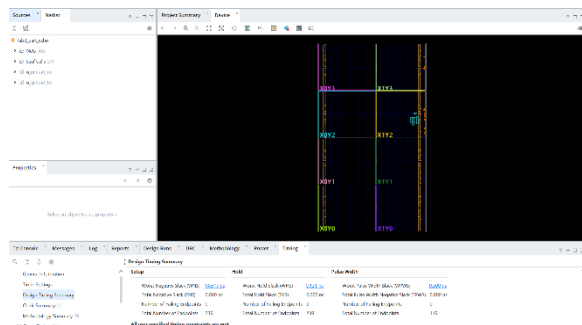


그림 53 : 실험 25 타이밍 분석 결과

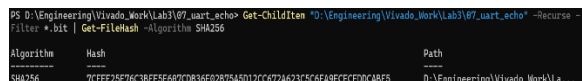


그림 54 : 실험 25 Bit Hash 확인

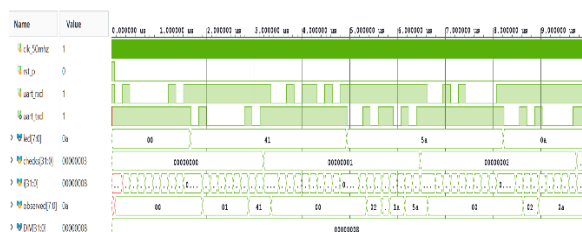


그림 55: 실험 25 입력 조합에 따른 출력 파형

Vivado 시뮬레이션 및 파형 비교

파형에서 uart_rxd 0x41, 0x5A, 0x0A가 순서대로 입력된 후, 각 데이터가 uart_txd를 통해 동일한 값으로 다시 전송되는 것을 확인하였다. 즉, RX가 한 프레임을 정상적으로 수신한 뒤 TX가 같은 byte를 echo하였다. LED 출력에는 마지막으로 수신한 데이터가 유지되었다.

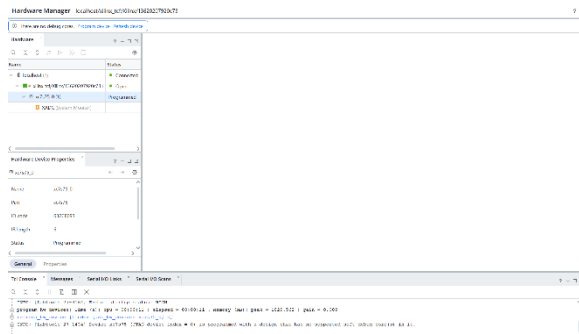


그림 56: 실험 25 Program Device 확인

FPGA 보드 시연

구체적인 보드 시연결과는 11절의 사진 및 시연 영상에서 자세히 분석하였다.

11. FPGA 시연 · 실험 25

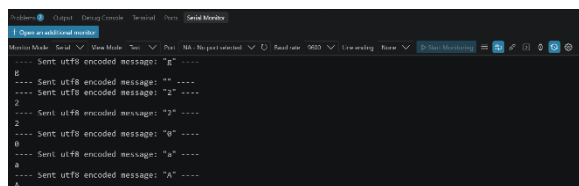


그림 57: 실험 25 FPGA 보드 시연 1

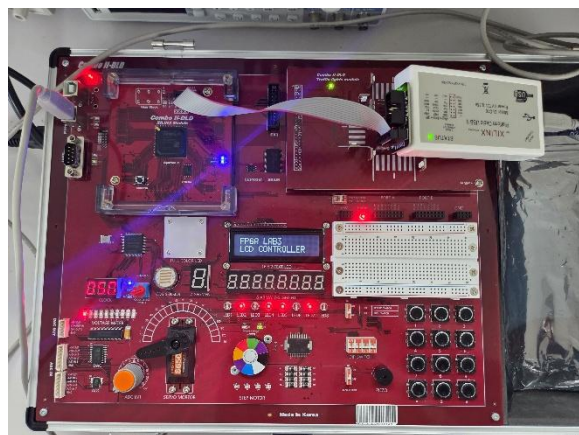


그림 58: 실험 25 FPGA 보드 시연 2

실험 25에서는 PC의 Serial Monitor와 FPGA 보드를 USB 직렬 연결로 연결하였다. Serial Monitor 9600 baud,

8N1, line ending None 및 local echo Off로 설정하였다.

시연 사진1을 보면 PC에서 g, 2, 0, a, A 등의 문자를 전송하자 FPGA가 해당 문자를 수신한 뒤 동일한 문자를 다시 송신하였다. Serial Monitor에서 송신한 각 문자 아래에 동일한 수신 문자가 표시되어 echo 동작을 확인하였다.

마지막으로 시연 사진2를 보면 Z 문자를 전송하였을 때 수신한 문자의 ASCII 값이 보드의 8개 일반 LED에 이진수로 표시되었다. Z의 아스키코드 값이 16진수로 5A이므로 왼쪽 LED 4개가 이진수로 5를, 오른쪽 4개가 이진수로 A(10)를 정확하게 표시하고 있다.

따라서 USB 직렬 연결, Serial Monitor의 송수신 결과 및 일반 LED 출력을 결과로 확인하였고, 영상을 증빙자료로 남겼다.

시연 영상: [실험25 시연 영상](#)

12. 그 외 실험별 시연 사진

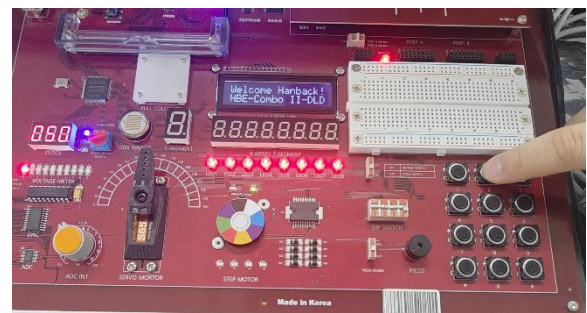


그림 59: 실험 19 FPGA 보드 시연

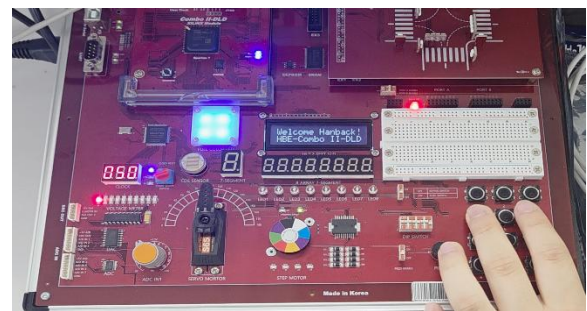


그림 60: 실험 20 FPGA 보드 시연

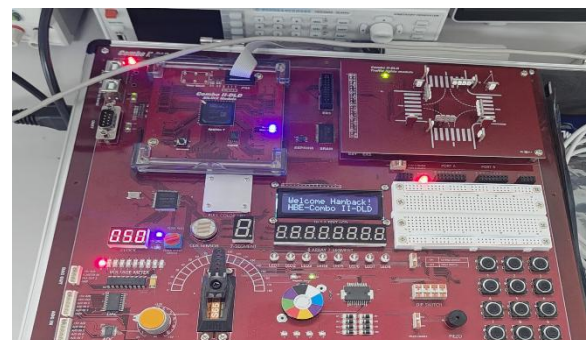


그림 61: 실험 21 FPGA 보드 시연

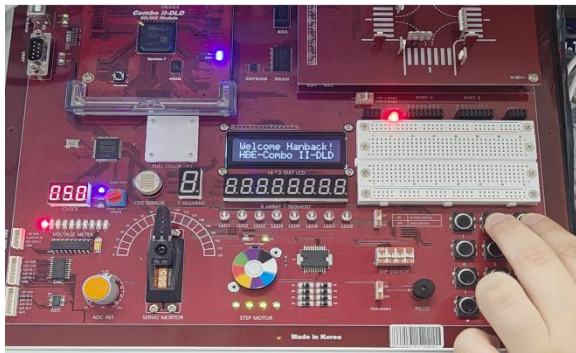


그림 62: 실험 22 FPGA 보드 시연

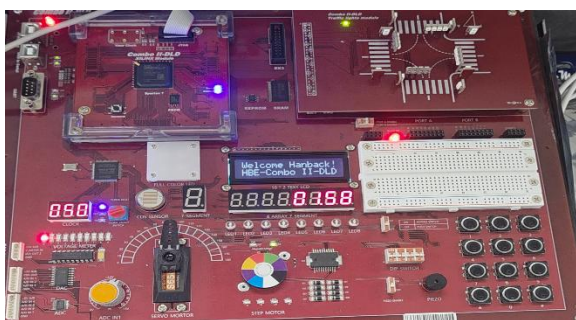


그림 63: 실험 23 FPGA 보드 시연

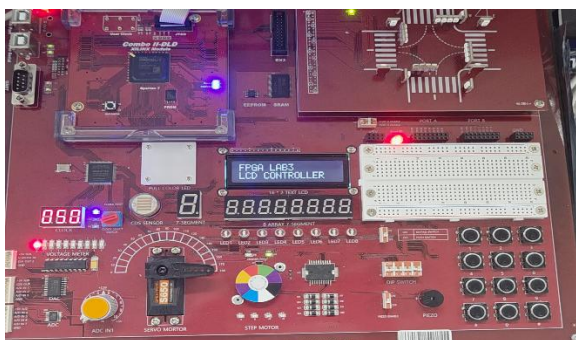


그림 64: 실험 24 FPGA 보드 시연

13. 문제 발생 및 해결 과정

VS Code와 Vivado의 파형 시간 단위 차이

동일한 시뮬레이션 파형을 확인할 때 VaporView에서는 시간축이 ps 단위로 표시되고 Vivado에서는 ns 또는 μs 단위로 표시되어 서로 다른 시간처럼 보이는 문제가 있었다. 이는 시뮬레이션 동작시간의 차이가 아니라 파형 뷰어가 확대 배율에 따라 선택한 표시 단위의 차이였다. 따라서 단위 환산 후 비교하여 시뮬레이션에 시간 오류가 없음을 하나씩 대조하여 확인하였다.

UART 연결 단자 문제와 USB 직렬 연결

실험 25에서 사용한 노트북에는 직접 연결할 수 있는 물

리적인 UART 또는 COM 단자가 없었다. 따라서 노트북의 USB 단자를 이용하는 USB 직렬 연결을 사용하였다. USB-to-UART 인터페이스는 PC에서 USB Serial Port인 가상 COM 포트로 인식되며, USB 통신을 보드에서 사용하는 UART 송수신 신호로 변환한다.

14. 예상 값 · Vivado · 보드 결과 비교

실험	TB검사 수	Vivado 결과	보드시연	최종
19	4 checks	VS Code동일	확인	정상
20	2 checks	VS Code동일	확인	정상
21	5 edges	VS Code동일	확인	정상
22	8 checks	VS Code동일	확인	정상
23	7 checks	VS Code동일	확인	정상
24	40 bytes	VS Code동일	확인	정상
25	3 checks	VS Code동일	확인	정상

TB에서 설정한 예상 값과 Vivado XSim의 출력 및 파형을 비교한 결과 모든 실험이 기대한 동작과 일치하였다. 실제 FPGA 보드에서도 각 실험의 주요 입출력 동작을 확인하였다. 실험 23의 59:59→00:00 처럼 실제 보드에서 장시간 확인이 필요한 항목은 가속 TB로 검증하고, 보드에서는 직접 확인할 수 있는 출력 동작을 검증하였다.

15. Bitstream 해시 표

실험	Bitstream	SHA-256
19	lab3_19_led_pwm.bit	219E50A...
20	lab3_20_rgb_pwm.bit	BE45B8C...
21	lab3_21_piezo.bit	8738A59...
22	lab3_22_stepper.bit	4E4CCC8...
23	lab3_23_mmss_clock.bit	F259C09...
24	lab3_24_character_lcd.bit	2CFB886...
25	lab3_25_uart_echo.bit	7CFEE25...

16. 결론

본 실험을 통해 PWM에서는 HIGH 시간의 비율이 LED의 평균 밝기를 결정하고, 피에조-스텝모터-시계-LCD-UART 회로에서는 카운터 값과 상태 전이 순서가 실제 출력의 주기와 동작을 결정하였다. 특히 UART와 LCD처럼 외부장치와 데이터를 주고받는 회로는 단순히 최종 출력만 확인하는 것이 아니라 제어신호, 데이터 순서 및 타이밍을 함께 검증해야 함을 알게 되었다.

또한 시뮬레이션에서는 가속 파라미터를 사용하여 긴 동작시간을 줄일 수 있지만, 실제 보드의 시간과 동일하지는 기본 파라미터와 타이밍 계산을 통해 별도로 확인해야 한다. 모든 회로가 Vivado의 타이밍 제약을 만족하고 실제 보드에서도 정상 동작한 결과를 통해, RTL 기능 검증과 FPGA 구현 결과, 실제 하드웨어 동작은 모두 함께 확인되어야 실험의 정확성과 완성도가 올라가고, 꼭 필요한 과정

임을 알게 되었다.

참고 문헌

[1] M. Morris Mano, Charles R. Kime, and Tom Martin, Logic and Computer Design Fundamentals, 5th ed., Pearson, 2016.